

Desarrollo Fullstack I - 003D

Informe Experiencia N°3
Proyecto PROPER: Sistema de Gestión de
Ventas de Productos de Aseo

Integrantes:

Josefina Isidora Pérez Huerta

Benjamín Elías Pérez Alfaro

ÍNDICE

Contexto del caso	3
Solución Planteada	4
Modelo de Bases de Datos y Modelo de Arquitectura	5
Servicio de Autenticación	6
I. ¿En qué consiste?	6
II. ¿Cómo se implementa la autenticación por token?	6
III. Spring security, protección de rutas, CORS	7
Implementación por Capas	8
I. Evidencia en Postman	9
II. Comunicación entre servicios: ¿Cómo se implementa?	11
Swagger	12
I. ¿Qué es? ¿En qué consiste?	12
II. ¿Cómo se implementa?	12
III. Ejemplo con evidencia	12
Testing	14
I. ¿En qué consiste? ¿Por qué es importante testear?	14
II. Explicar en qué consisten las pruebas unitarias (AAA)	14
III. Ejemplo con evidencia	15

Contexto del caso

Proper es una empresa dedicada a la fabricación, comercialización y distribución de productos de limpieza orientados a diversos sectores, entre ellos el hogar, la industria automotriz, la sanitización y el tratamiento de pisos. Su actividad se centra en ofrecer soluciones integrales de limpieza mediante una amplia variedad de productos y líneas especializadas, dirigidas tanto a empresas como a clientes particulares.

Actualmente, la organización presenta diversas dificultades asociadas a la gestión de sus procesos comerciales y operacionales. Entre las principales problemáticas identificadas se encuentran la administración manual y desorganizada de pedidos y ventas, la dificultad para gestionar eficientemente la información relacionada con clientes, vendedores y productos, y la ausencia de una plataforma centralizada para el almacenamiento y consulta de datos.

Asimismo, se evidencian retrasos en el cálculo de bonificaciones para los vendedores, un alto riesgo de errores derivados del registro manual de pedidos y montos de venta, y limitaciones para escalar el sistema de gestión conforme aumentan las líneas de productos y las áreas de negocio de la empresa.

Solución Planteada

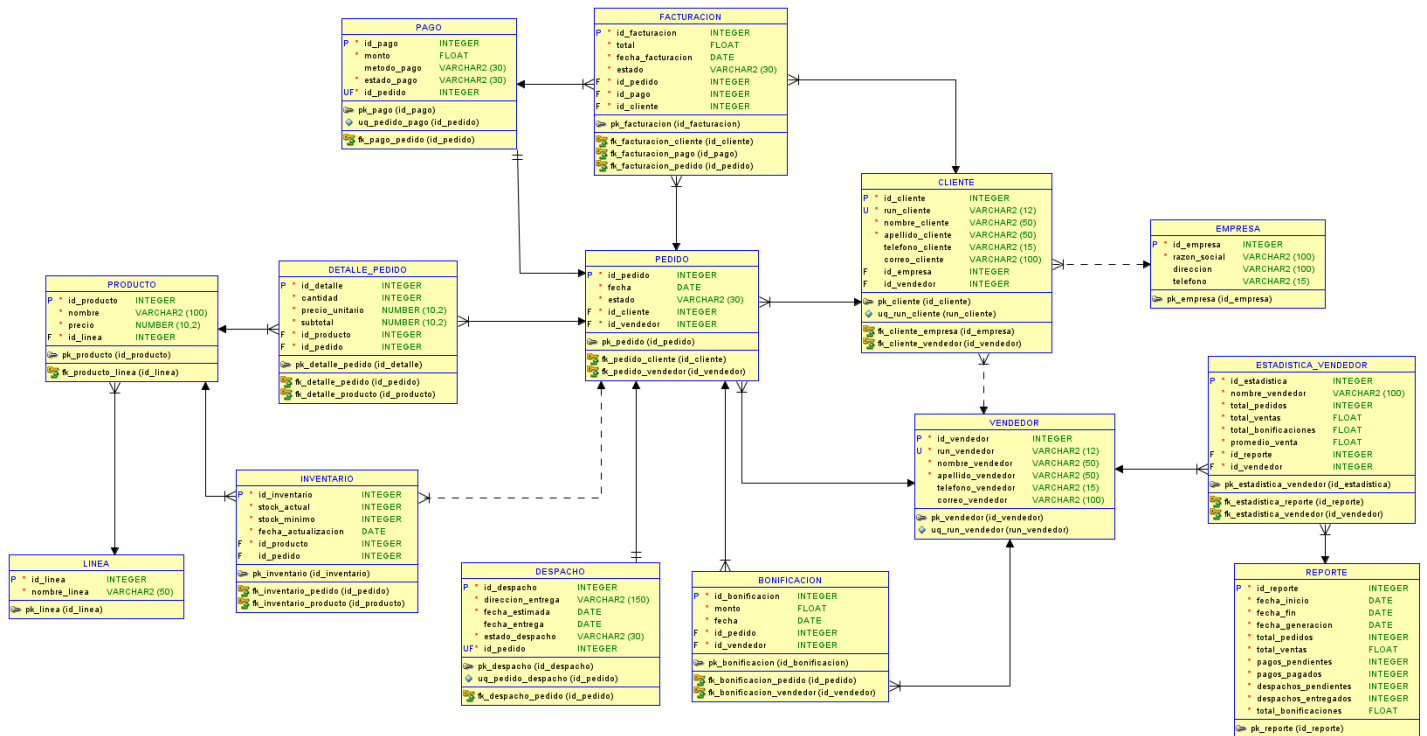
Con el objetivo de abordar las problemáticas identificadas, se propone el desarrollo e implementación de una arquitectura basada en microservicios, diseñada para proporcionar una solución escalable, mantenible y eficiente para la gestión de los procesos de venta de la empresa.

La solución fue desarrollada utilizando el framework Spring Boot y gestionada mediante Maven, permitiendo una adecuada administración de dependencias y una estructura organizada del proyecto. Cada microservicio fue implementado de forma independiente, contando con su propia base de datos y un puerto de ejecución específico, garantizando así un bajo acoplamiento y una mayor autonomía entre los distintos componentes del sistema.

La comunicación entre los microservicios se realizó mediante WebClient, herramienta que facilita el intercambio de información y la integración de funcionalidades entre los diferentes servicios. Adicionalmente, se incorporó un API Gateway como punto único de acceso al sistema, centralizando la gestión de rutas y optimizando el control de las solicitudes realizadas por los usuarios y aplicaciones consumidoras.

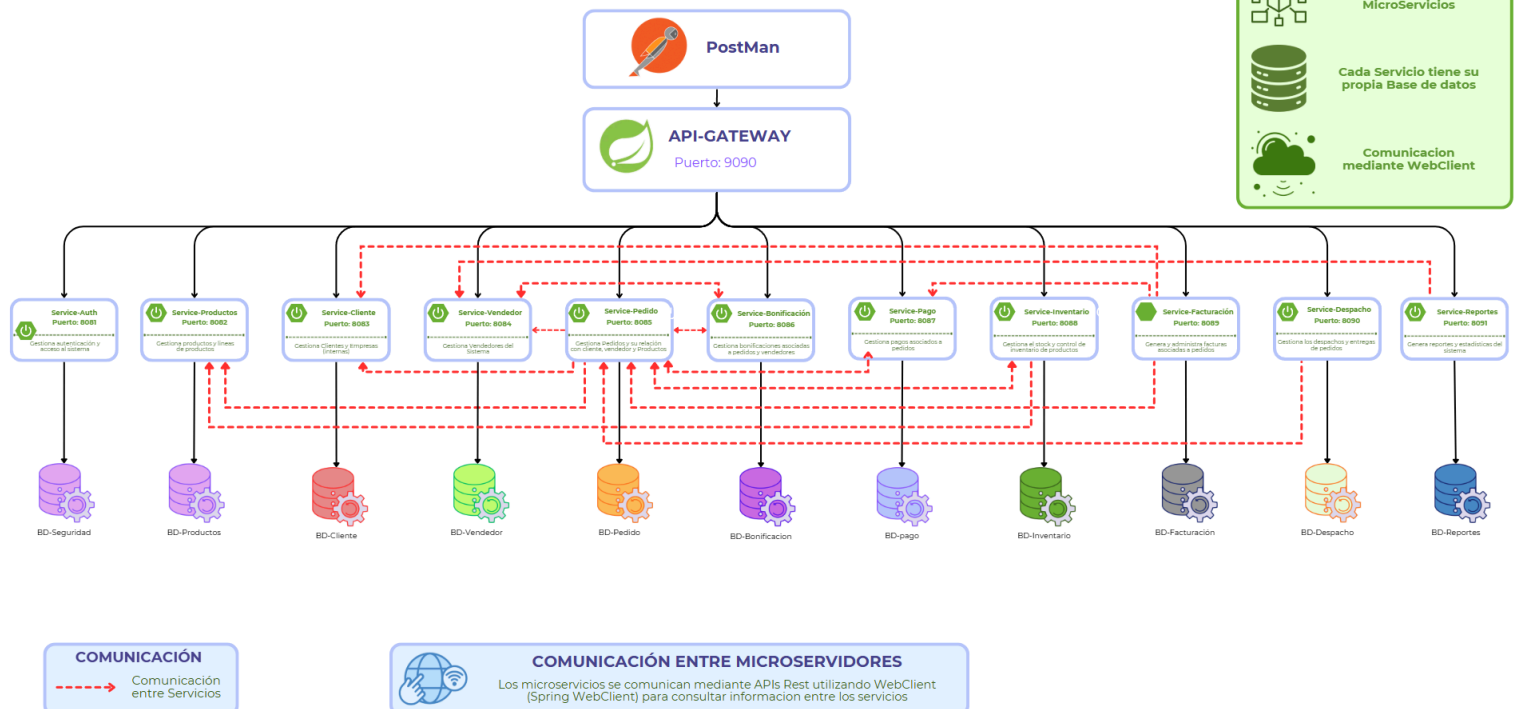
Esta arquitectura permite mejorar la organización de la información, reducir los errores asociados a procesos manuales y proporcionar una base tecnológica capaz de adaptarse al crecimiento futuro de la empresa.

Modelo de Bases de Datos y Modelo de Arquitectura



ARQUITECTURA DE MICROSERVICIOS

Sistema de Gestión de Ventas de Productos de Aseo



Servicio de Autenticación

I. ¿En qué consiste?

El servicio de autenticación es el componente encargado de validar la identidad de los usuarios que acceden al sistema. Su principal función es verificar que las credenciales proporcionadas por el usuario sean correctas antes de permitir el acceso a los recursos protegidos de la aplicación.

Dentro de una arquitectura basada en microservicios, la autenticación se implementa de manera centralizada mediante un microservicio dedicado, permitiendo separar las responsabilidades de seguridad del resto de la lógica de negocio. Esto mejora la escalabilidad, el mantenimiento y la seguridad general de la plataforma.

En nuestro proyecto se implementó un microservicio denominado service-auth, encargado del registro de usuarios, validación de credenciales y generación de tokens de acceso.

II. ¿Cómo se implementa la autenticación por token?

La autenticación se implementó utilizando JWT (JSON Web Token). Este mecanismo permite validar la identidad de los usuarios sin necesidad de almacenar sesiones en el servidor.

El flujo de autenticación es el siguiente:

1. El usuario se registra mediante el endpoint /auth/registrar.
2. La contraseña es cifrada utilizando el algoritmo BCrypt antes de almacenarse en la base de datos.
3. Posteriormente el usuario inicia sesión mediante el endpoint /auth/login.
4. El sistema valida las credenciales ingresadas.
5. Si la validación es correcta, se genera un token JWT firmado digitalmente.
6. El token es enviado al cliente.
7. En las solicitudes posteriores, el cliente debe incluir el token para acceder a los recursos protegidos.

III. Spring security, protección de rutas, CORS

Para reforzar la seguridad de la aplicación se utilizó Spring Security, framework especializado en autenticación y autorización dentro del ecosistema Spring.

Las rutas correspondientes al registro e inicio de sesión fueron configuradas como públicas mediante:

```
.requestMatchers("/auth/**").permitAll()
```

Mientras que el resto de los endpoints requieren autenticación previa.

Además, se configuró el manejo de CORS (Cross-Origin Resource Sharing), permitiendo controlar qué aplicaciones externas pueden consumir las APIs del sistema y evitando accesos no autorizados desde dominios desconocidos.

Esta configuración permite garantizar que únicamente los usuarios autenticados puedan acceder a los recursos protegidos del sistema.

Implementación por Capas

El desarrollo de nuestros microservicios se rige por la arquitectura de diseño por capas, lo que permite un desacoplamiento de responsabilidades, facilitando así el mantenimiento y el testeo unitario independiente.

Cada microservicio se estructura principalmente en las siguientes capas:

Controller: Corresponde a la capa de presentación. Su responsabilidad es recibir las solicitudes HTTP provenientes de los clientes, validar los datos de entrada y delegar la lógica de negocio a la capa de servicios.

Service: Contiene la lógica de negocio de la aplicación. En esta capa se realizan validaciones, cálculos, reglas comerciales y comunicación con otros microservicios.

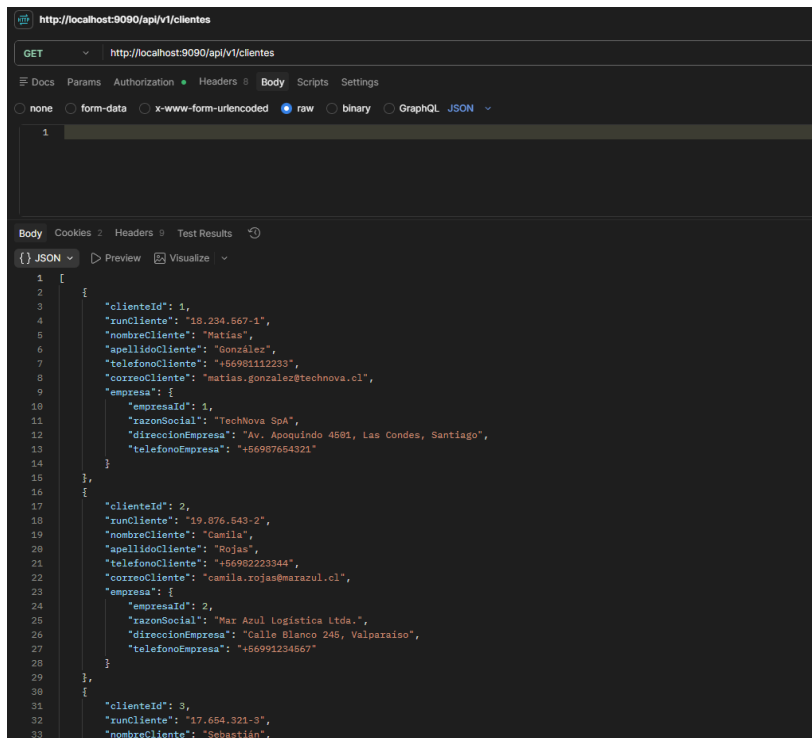
Repository: Es la capa encargada del acceso a datos. Utiliza Spring Data JPA para interactuar con las bases de datos y realizar operaciones CRUD sobre las entidades.

Model: Representa las entidades del dominio del negocio. Cada entidad es mapeada a una tabla dentro de la base de datos mediante JPA.

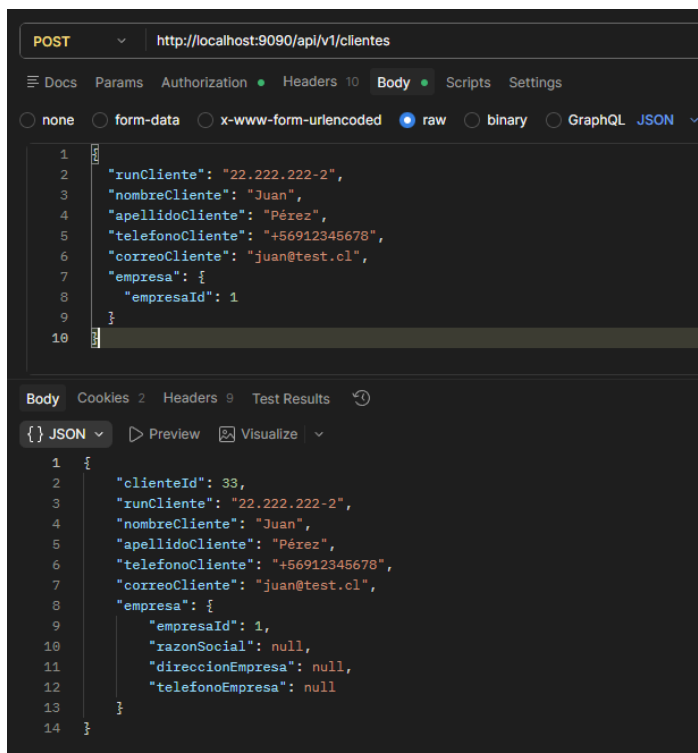
Esta separación permite una arquitectura más ordenada, reutilizable y fácil de mantener a largo plazo.

I. Evidencia en Postman

GET (para buscar a todos los clientes)



POST



GET (con id cliente)

GET <http://localhost:9090/api/v1/clientes/33>

Docs Params Authorization Headers 10 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "runCliente": "22.222.222-2",
3   "nombreCliente": "Juan",
4   "apellidoCliente": "Pérez",
5   "telefonoCliente": "+56912345678",
6   "correoCliente": "juan@test.cl",
7   "empresa": {
8     "empresaId": 1
9   }
10 }
```

Body Cookies 2 Headers 9 Test Results

{ } JSON Preview Visualize

```
1 {
2   "clienteId": 33,
3   "runCliente": "22.222.222-2",
4   "nombreCliente": "Juan",
5   "apellidoCliente": "Pérez",
6   "telefonoCliente": "+56912345678",
7   "correoCliente": "juan@test.cl",
8   "empresa": {
9     "empresaId": 1,
10    "razonSocial": "TechNova SpA",
11    "direccionEmpresa": "Av. Apoquindo 4501, Las Condes, Santiago",
12    "telefonoEmpresa": "+56987654321"
13  }
14 }
```

PUT

PUT <http://localhost:9090/api/v1/clientes/33>

Docs Params Authorization Headers 10 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

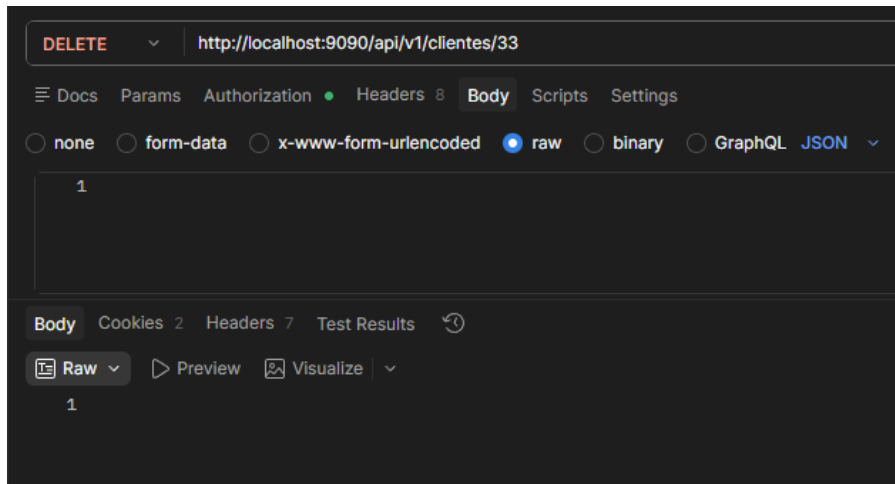
```
1 {
2   "runCliente": "22.222.222-2",
3   "nombreCliente": "Juan Carlos",
4   "apellidoCliente": "Pérez",
5   "telefonoCliente": "+56987654321",
6   "correoCliente": "juancarlos@test.cl",
7   "empresa": {
8     "empresaId": 1
9   }
10 }
```

Body Cookies 2 Headers 9 Test Results

{ } JSON Preview Visualize

```
1 {
2   "clienteId": 33,
3   "runCliente": "22.222.222-2",
4   "nombreCliente": "Juan Carlos",
5   "apellidoCliente": "Pérez",
6   "telefonoCliente": "+56987654321",
7   "correoCliente": "juancarlos@test.cl",
8   "empresa": {
9     "empresaId": 1,
10    "razonSocial": null,
11    "direccionEmpresa": null,
12    "telefonoEmpresa": null
13  }
14 }
```

DELETE



II. Comunicación entre servicios: ¿Cómo se implementa?

La comunicación entre microservicios se implementó utilizando **Spring WebFlux WebClient**.

Este mecanismo permite que un microservicio pueda consumir endpoints de otro servicio de forma programática, intercambiando información necesaria para completar los procesos de negocio.

Por ejemplo, cuando se genera un pedido, el microservicio de pedidos se comunica con otros servicios para:

- Generar automáticamente la bonificación correspondiente al vendedor.
- Crear el pago asociado al pedido.
- Actualizar el inventario descontando el stock disponible.
- Generar el despacho asociado a la venta.

Esta comunicación es de tipo síncrona, ya que el proceso principal depende de la respuesta exitosa de los servicios involucrados para garantizar la consistencia de los datos.

El uso de WebClient permitió mantener una arquitectura desacoplada, donde cada microservicio conserva su independencia funcional y de base de datos.

Swagger

I. ¿Qué es? ¿En qué consiste?

Swagger es una suite de herramientas de software de código abierto diseñada para describir, producir, consumir y documentar de forma interactiva y automatizada los endpoints de las APIs.

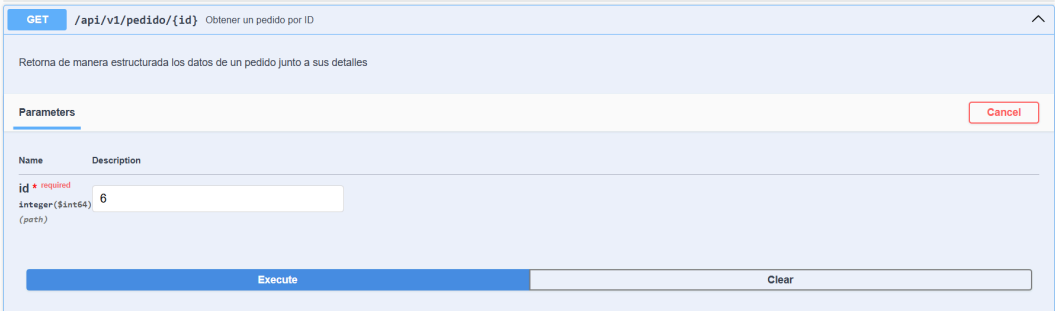
Consiste en escanear el código fuente de Spring Boot en tiempo de ejecución para autogenerar una especificación estándar en formato JSON o YAML. A partir de esa especificación, expone su interfaz visual interactiva basada en web donde personas tales como desarrolladores, testers o clientes pueden visualizar todos los métodos disponibles, con sus parámetros requeridos, esquemas de entrada/salida y realizar peticiones de prueba reales sin necesidad de abrir herramientas externas como Postman.

II. ¿Cómo se implementa?

Para incorporar swagger en nuestros microservicios de Spring Boot, seguimos los siguientes pasos: primero se agrega la dependencia oficial en el archivo pom.xml en cada microservicio que tenga el proyecto, para después agregar notaciones como @Tag, @Operation, @ApiResponse, dentro del controlador y así finalmente ingresar desde el navegador a la dirección dinámica generada.

III. Ejemplo con evidencia

GET PEDIDO



```
50 @GetMapping("/{id}")
51 @Operation(summary = "Obtener un pedido por ID", description = "Retorna de manera estructurada los datos de un pedido junto a sus detalles")
52 @ApiResponse(responseCode = "200", description = "Pedido encontrado con éxito")
53 @ApiResponse(responseCode = "404", description = "El pedido solicitado no fue localizado")
54 public Pedido verPedido(@PathVariable Long id)
55 {
56     return pedidoService.obtenerPedidoPorId(id);
57 }
```

POST PRODUCTOS

POST /api/v1/productos Crear un nuevo producto

Registra un producto en el sistema y retorna el objeto creado

Parameters

CancelReset

No parameters

Request body *required*

application/json

```
{  "productoId": 1,  "productoNombre": "Shampoo canino supercachupin 3000",  "precio": 3000.0,  "linea": {    "lineaId": 6,    "nombre": "Linea Cachupin"  }}
```

Execute

Server response

Code	Details
200	Response body<pre>{ "productoId": 1, "productoNombre": "Shampoo canino supercachupin 3000", "precio": 3000.0, "linea": { "lineaId": 6, "nombre": "Linea Cachupin" }}</pre> Download Response headers<pre>content-type: application/json</pre>

```
61 @PostMapping
62 @Operation(summary = "Crear un nuevo producto", description = "Registra un producto en el sistema y retorna el objeto creado")
63 @ApiResponse(responseCode = "200", description = "Producto creado con éxito")
64 @ApiResponse(responseCode = "400", description = "Datos mal formados o faltantes")
65 public ResponseEntity<Producto> crear(@Valid @RequestBody Producto producto)
66 {
67     return ResponseEntity.ok(productoService.guardarProducto(producto));
68 }
```

DELETE CLIENTE

DELETE /api/v1/clientes/{id} Eliminar un cliente por ID

Remueve permanentemente a un cliente de la base de datos

Parameters

Cancel

Name	Description
id <i>required</i>	
integer(\$int64)	8
(path)	

ExecuteClear

Responses

Curl

Server response

Code	Details	
204		
Responses		
Code	Description	Links
204	Cliente eliminado con éxito	No links

```
60 @DeleteMapping("/{id}")
61 @Operation(summary = "Eliminar un cliente por ID", description = "Remueve permanentemente a un cliente de la base de datos")
62 @ApiResponse(responseCode = "204", description = "Cliente eliminado con éxito")
63 public ResponseEntity<Void> eliminarCliente(@PathVariable Long id)
64 {
65     clienteService.eliminarCliente(id);
66     return ResponseEntity.noContent().build();
67 }
```

Testing

I. ¿En qué consiste? ¿Por qué es importante testear?

El testing es el primer nivel de pruebas del sistema, se centra principalmente en pequeños códigos de forma aislada ya sea una función, un método o una clase. El objetivo principal es verificar que el sistema cumpla con los requisitos mínimos para su funcionamiento y asegurar que el software para el usuario final sea el esperado.

Es importante testear por que crucial tanto para la reducción de costos ya que detectar un error al inicio es mas barato que tener que solucionarlo cuando el sistema esta en producción, la seguridad de datos de los usuarios es muy importante debido a que es una función crítica para un negocio y al probar el sistema se utilizan doble para poder simular escenarios y respuesta de forma rápida y efectiva.

II. Explicar en qué consisten las pruebas unitarias (AAA)

Principalmente las pruebas unitarias se enfocan en validar de forma aislada un método o clase. Para mantener estas pruebas ordenadas y limpias se utilizan el patrón AAA:

- Arrange (Organizar/Preparar): Principalmente es esa fase se inicializan las variables, se configuran los datos de entrada y crean los objetos simulados que se necesitan operar.
- Act (Actual/Ejecutar): Es la ejecución real del método o función que se está sometiendo a prueba, pasando por la configuración anterior
- Assert(Afirmar/Verificar): es la fase de validación. En al cual se contrasta el resultado obtenido en la ejecución contra el resultado esperado. Si coinciden la prueba pasa, si no, falla.

III. Ejemplo con evidencia

The image displays two screenshots of an IDE, likely IntelliJ IDEA, showing Java code for unit tests. The top screenshot shows the 'ClienteServiceTest.java' file, and the bottom screenshot shows the 'ReporteServiceTest.java' file. Both files are part of a project named 'com.proper.service_cliente'.

Top Screenshot: ClienteServiceTest.java

```
1 package com.proper.service_cliente.service;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertNotNull;
5 import static org.mockito.ArgumentMatchers.any;
6 import static org.mockito.Mockito.times;
7 import static org.mockito.Mockito.verify;
8 import static org.mockito.Mockito.when;
9
10 import org.junit.jupiter.api.DisplayName;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15 import org.mockito.junit.jupiter.MockitoExtension;
16
17 import com.proper.service_cliente.model.Cliente;
18 import com.proper.service_cliente.repository.ClienteRepository;
19 import com.proper.service_cliente.repository.EmpresaRepository;
20
21 @ExtendWith(MockitoExtension.class) // Usamos Mockito para simular objetos
22 public class ClienteServiceTest {
23     @Mock
24     private ClienteRepository clienteRepository;
25     @Mock
26     private EmpresaRepository empresaRepository;
27     @InjectMocks
28     private ClienteService clienteService;
29     @Test
30     @DisplayName("Deberia guardar un cliente correctamente")
31     void guardarClienteTest() {
32         Cliente cliente = new Cliente();
33         cliente.setNombreCliente(nombreCliente: "Manuel");
34         cliente.setRunCliente(runCliente: "12333232-4");
35         when(clienteRepository.save(any(type: Cliente.class))).thenReturn(invocation->{
36             Cliente c=invocation.getArgument(index: 0);
37             c.setClientId(clienteId: 1L);
38             return c;
39         });
40     }
```

Bottom Screenshot: ReporteServiceTest.java

```
1 package com.proper.service_reporte.service;
2
3 import static org.mockito.ArgumentMatchers.anyString;
4 import static org.mockito.Mockito.times;
5 import static org.mockito.Mockito.verify;
6 import static org.mockito.Mockito.when;
7
8 import java.time.LocalDate;
9 import java.util.List;
10 import java.util.Map;
11
12 import org.junit.jupiter.api.Test;
13 import org.junit.jupiter.api.extension.ExtendWith;
14 import org.mockito.InjectMocks;
15 import org.mockito.Mock;
16 import org.mockito.junit.jupiter.MockitoExtension;
17 import org.springframework.web.reactive.function.client.WebClient;
18
19 import com.proper.service_reporte.model.Reporte;
20 import com.proper.service_reporte.repository.ReporteRepository;
21
22 import reactor.core.publisher.Mono;
23
24 @ExtendWith(MockitoExtension.class)
25 public class ReporteServiceTest {
26     @Mock
27     private ReporteRepository reporteRepository;
28     @Mock
29     private WebClient.Builder webClientBuilder;
30     @InjectMocks
31     private ReporteService reporteService;
32
33     @Test
34     void generarReporteTest() {
35         // 1. PREPARACION ---
36         LocalDate inicio = LocalDate.of(year: 2026, month: 6, dayOfMonth: 1);
37         LocalDate fin = LocalDate.of(year: 2026, month: 6, dayOfMonth: 30);
38         Long pedidoId = 99L;
39         Long vendedorId = 7L;
40
41         // Mapas de prueba simulando lo que entregan los otros microservicios
42         Map<String, Object> detalle = Map.of(k1: "subtotal", v1: 150.0);
43         List<Map<String, Object>> listaPedidos = List.of(Map.of(
44             k1: "pedidoId", pedidoId, k2: "vendedorId", vendedorId, k3: "fecha", v3: "2026-06-15", k4: "detalles", List.of(detalle)
45         ));
46         List<Map<String, Object>> listaPagos = List.of(Map.of(
47             k1: "pedidoId", pedidoId, k2: "monto", v2: 150.0, k3: "estadoPago", v3: "PAGADO"
48         ));
49     }
```

EXPLORER

PROPER-PEREZ-PEREZ-003D

OUTLINE

TIMELINE

LOGICAL STRUCTURE

JAVA PROJECTS

api-gateway

service-auth

service-bonificacion

service-cliente

service-despacho

src/main/java

com.proper.service_despacho

com.proper.service_despacho.config

com.proper.service_despacho.controller

com.proper.service_despacho.exception

com.proper.service_despacho.model

Despacho

PedidoDTO

com.proper.service_despacho.repository

com.proper.service_despacho.service

src/main/resources

src/test/java

com.proper.service_despacho

com.proper.service_despacho.service

DespachoServiceTest

target/generated-sources/annotations

target/generated-test-sources/test-annotations

JRE System Library [JavaSE-21]

Maven Dependencies

mvn

src

target

gitattributes

gitignore

mvnw

mvnw.cmd

pom.xml

service-facturacion

service-inventario

service-pago

service-pedido

service-productos

service-report

service-vendedor

DespachoServiceTest.java 1,0 X

service-despacho > src > test > java > com > proper > service_despacho > service > DespachoServiceTest.java > ...

```
1 package com.proper.service_despacho.service;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertNotNull;
5 import static org.junit.jupiter.api.Assertions.assertTrue;
6 import static org.mockito.ArgumentMatchers.anyString;
7 import static org.mockito.Mockito.times;
8 import static org.mockito.Mockito.verify;
9 import static org.mockito.Mockito.when;
10
11 import java.util.Optional;
12
13 import org.junit.jupiter.api.DisplayName;
14 import org.junit.jupiter.api.Test;
15 import org.junit.jupiter.api.extension.ExtendWith;
16 import org.mockito.InjectMocks;
17 import org.mockito.Mock;
18 import org.mockito.Mockito;
19 import org.mockito.junit.jupiter.MockitoExtension;
20 import org.springframework.web.reactive.function.client.WebClient;
21
22 import com.proper.service_despacho.model.Despacho;
23 import com.proper.service_despacho.model.PedidoDTO;
24 import com.proper.service_despacho.repository.DespachoRepository;
25
26 import reactor.core.publisher.Mono;
27
28 @ExtendWith(MockitoExtension.class)
29 public class DespachoServiceTest {
30     @Mock
31     private DespachoRepository despachoRepository; // Simulamos la BD local de despachos
32
33     @Mock
34     private WebClient.Builder webClientBuilder; // Simulamos el cliente HTTP
35
36     @InjectMocks
37     private DespachoService despachoService; // Inyectamos los mocks en el servicio real
38
39     @Test
40     @DisplayName("Deberia buscar un despacho por ID y enriquecerlo con su Pedido usando DTOs")
41     void buscarPorIdConEnriquecimientoPedidoTest() {
42         // --- 1. PREPARACIÓN (Escenario) ---
43         Long despachoId = 1L;
44         Long pedidoId = 202L;
45
46         // Instanciamos el objeto local Despacho simulado
47         Despacho despachoMock = new Despacho();
48         despachoMock.setDespachoId(despachoId);
49         despachoMock.setPedidoId(pedidoId);
50         despachoMock.setEstadoDespacho(estadoDespacho: "EN_RUTA");
51
52         // Instanciamos el DTO simulado con los datos externos del Pedido
53         PedidoDTO pedidoMockExternal = new PedidoDTO(pedidoId, estado: "PAGADO", total: 45000.0);
54
55         // Configuramos el comportamiento del repositorio local
56         when(despachoRepository.findById(despachoId)).thenReturn(Optional.of(despachoMock));
57     }
```

EXPLORER

PROPER-PEREZ-PEREZ-003D

OUTLINE

TIMELINE

LOGICAL STRUCTURE

JAVA PROJECTS

api-gateway

service-auth

service-bonificacion

service-cliente

service-despacho

service-facturacion

service-inventario

service-pago

service-pedido

service-productos

service-report

service-vendedor

src/main/java

src/main/resources

src/test/java

com.proper.service_vendedor

com.proper.service_vendedor.service

VendedorServiceTest

target/generated-sources/annotations

target/generated-test-sources/test-annotations

JRE System Library [JavaSE-21]

Maven Dependencies

mvn

src

target

gitattributes

gitignore

mvnw

mvnw.cmd

pom.xml

VendedorServiceTest.java X

service-vendedor > src > test > java > com > proper > service_vendedor > service > VendedorServiceTest.java > VendedorServiceTest > guardarVendedorTest()

```
1 You 2 hours ago | 1 author (You)
2 package com.proper.service_vendedor.service;
3
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5 import static org.mockito.ArgumentMatchers.any;
6 import static org.mockito.Mockito.times;
7 import static org.mockito.Mockito.verify;
8 import static org.mockito.Mockito.when;
9
10 import org.junit.jupiter.api.DisplayName;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15
16 import com.proper.service_vendedor.model.Vendedor;
17 import com.proper.service_vendedor.repository.VendedorRepository;
18
19 You 2 hours ago | 1 author (You)
20 @ExtendWith(MockitoExtension.class)//Usamos Mockito para simular objetos
21 class VendedorServiceTest {
22     @Mock
23     private VendedorRepository vendedorRepository;
24
25     @InjectMocks
26     private VendedorService vendedorService;
27
28     @Test
29     @DisplayName("Deberiamos guardar un vendedor correctamente")
30     void guardarVendedorTest() {
31         {
32             Vendedor vendedor = new Vendedor();
33             vendedor.setNombreVendedor(nombreVendedor: "Miguel");
34             vendedor.setRunVendedor(runVendedor: "10765555-2");
35
36             when(vendedorRepository.save(any(type: Vendedor.class))).thenReturn(invocation->{
37                 Vendedor v=invocation.getArgument(index: 0);
38                 v.setVendedorId(vendedorId: 1L);
39                 return v;
40             });
41         }
```